

Announcing Helidon Talon

OCI's next generation framework for building OCI Native Services

2.0.0 RC2 · July 2026

What Is Helidon Talon?

- Helidon 4 extensions for OCI Native Services
- Standardize config, authentication, OCI clients and observability

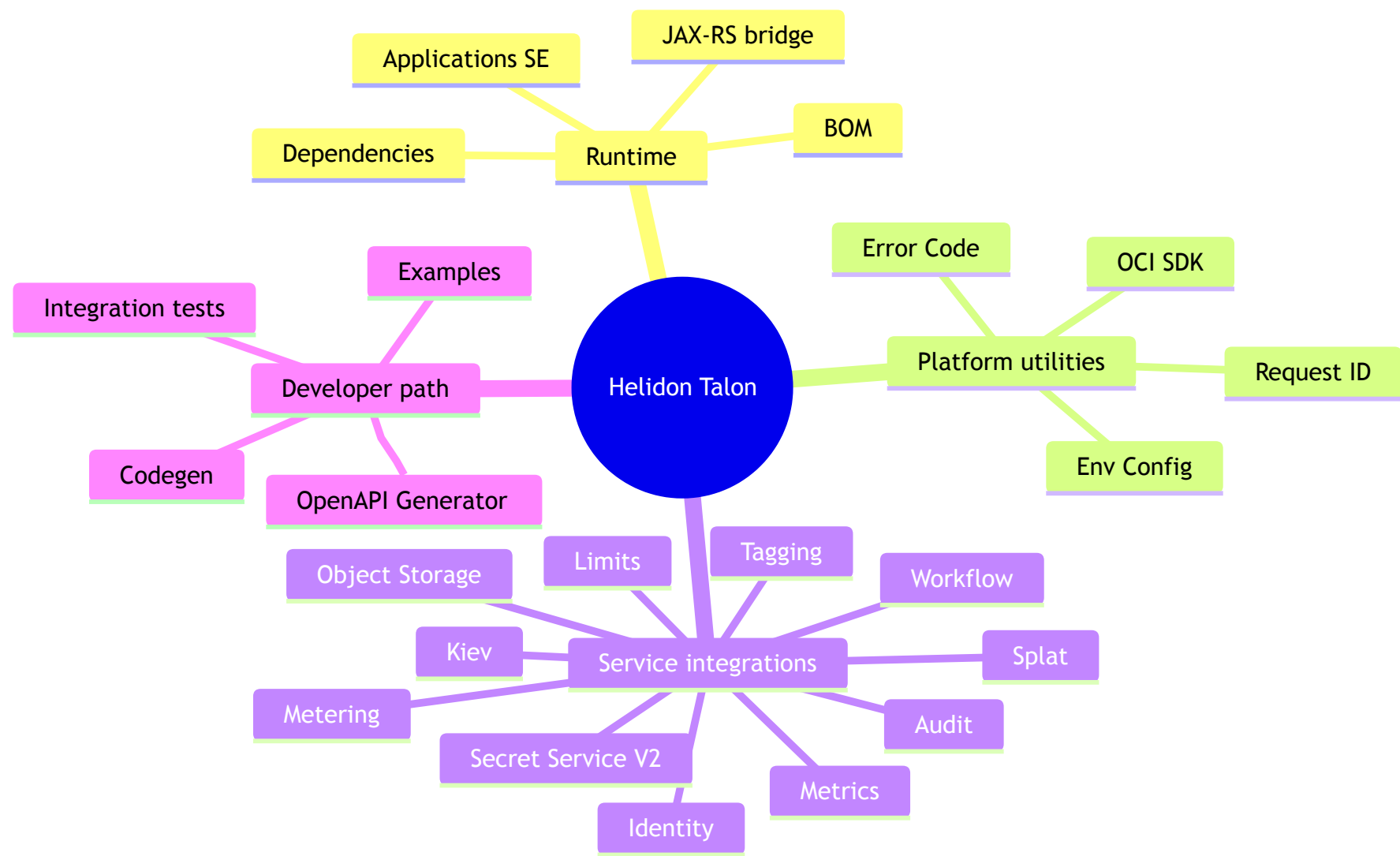
Build a New Service from Scratch

- Start with a Helidon 4 SE application
- Add only the OCI integrations the service needs
- Follow the reference app and runnable examples

Migrate a Dropwizard Application

- Use Heliport and Codex to guide the migration
- Replace repeated platform wiring with Helidon Talon modules
- Validate the application incrementally

At a Glance



Helidon Is Built for Modern Java Services

Helidon is open source and available at <https://helidon.io/>.

Java-First

- Java framework for microservices
- Early adoption of new JDK releases and Java features
- Lightweight runtime with strong performance

Developer-Friendly

- Code that is direct, readable, and easy to debug
- Imperative and declarative programming styles
- Build-time services without runtime scanning

Cloud-Native

- Designed for containers and Kubernetes
- REST, gRPC, GraphQL, and WebSocket APIs
- Observability and ecosystem integrations

Helidon 4 Makes Scale Simpler

Before Virtual Threads

- Platform threads limited concurrency
- Reactive pipelines improved scale
- Teams paid with a different coding and debugging model

With Helidon 4

- Each request runs on a lightweight virtual thread
- Blocking code scales without reactive plumbing
- Service logic stays straightforward and maintainable

Result: high concurrency with the Java programming model teams already know.

Helidon Talon Builds on the Helidon 4 Model

Programming Model

- Helidon SE-first services
- Compile-time injection via the service registry
- Typed configuration with generated metadata
- Direct service code with platform interceptors

Runtime Model

- JDK 25 baseline
- High-performance request handling with virtual threads
- Blocking code instead of async/reactive plumbing
- Helidon infrastructure for routing, lifecycle, config, and observability

Helidon SE Declarative

Service annotations define endpoint discovery, lifecycle, and injection at build time.

```
@RestServer.Endpoint
@Path("/greetings")
@Service.Singleton
class GreetingResource {
    private final GreetingService service;

    @Service.Inject
    GreetingResource(GreetingService service) {
        this.service = service;
    }
    ...
}
```

Helidon SE Declarative

HTTP annotations define routing, media types, and request binding.

```
@Http.GET
@Http.Path("/{name}")
@Http.Produces(MediaType.APPLICATION_JSON_VALUE)
Greeting get(@Http.PathParam("name") String name,
             @Http.QueryParam("lang") Optional<String> lang) {
    return service.get(name, lang.orElse("en"));
}

@Http.POST
@Http.Consumes(MediaType.APPLICATION_JSON_VALUE)
Greeting create(@Http.Entity Greeting value) {
    return service.create(value);
}
}
```


Talon Baseline and Size

Helidon 4

Target runtime, currently `4.5.1-M1`

30+

Helidon Talon artifacts in BOM

JDK 25

Compiler and documented baseline

13+

Documented service integrations

OCI SDK 2.88.0

Managed through dependencies

12+

Runnable example applications

OCI Environment Config

- Talon contributes a lazy `oci-env` source to Helidon Config
- Publishes region, AD, FD, and OCI domain names as `oci.env.*`
- Resolves location from OCI runtime files or IMDS and supports local/test overrides
- Auto-registers by default or as an added source when using meta-config

How `oci-env` Is Used

Discover

Region, AD, FD, public domain, and IaaS domain

Publish

Config keys such as
`oci.env.region-name`

Consume

Clients derive endpoints and SDK region services

- Keeps region and domain logic out of client integration code
- Explicit `sources[].properties` wins per key over `helidon.oci-env`
- Example: Secret Service can use `${oci.env.iaas-domain-name}`

OCI SDK Authentication

- Shared authentication selection for OCI Java SDK clients
- Exposes the selected `BasicAuthenticationDetailsProvider`
- Auth methods are contributed by classpath modules
- Client integrations consume one provider contract
- Helidon Talon adds `service-principal` to Helidon's available authentication methods

OCI SDK Authentication Configuration

- Config root is `helidon.oci`
- `authentication-method` selects or requires a provider
- `auto` tries available methods from the service registry
- `BasicAuthenticationDetailsProvider` can be injected

OCI Bootstrap Configuration Example

```
helidon:
  oci:
    authentication-method: "instance-principal"
    imds-timeout: "PT3S"
    imds-base-uri: http://localhost:8000/opc/v2/
    imds-detect-retries: 1
  oci-env:
    dynamic-core-regions:
      enabled: false
    location-override:
      region: us-ashburn-1
      availability-domain: iad-ad-1
      fault-domain: 1
```

Service Integrations

Filters + Interceptors

**Injectable Clients +
Config**

**Declarative API +
Codegen**

Helidon Service Registry

Identity

- Integrates Helidon endpoints with the OCI Auth SDK
- Provides authentication, authorization, and SPLAT-aware request handling
- Registers `Principal`, `AuthorizationRequest`, and `IdentityContext` per request
- Uses annotations and built-time generated interceptors

Identity Configuration

- Config root is `oci.identity`
- Authentication and authorization are configured independently
- Region and location defaults can come from `oci-env`

```
oci:
  identity:
    authentication:
      global-business-unit: "Cloud-Infra"
      team-name: "ExampleTeam"
      application-name: "MyService"
      use-instance-principal: true
    authorization:
      enabled: true
      service-name: "my-service"
```

Identity Example

- Auth SDK annotations drive generated authorization interceptors
- Request data can be injected directly into resource methods

```
@RestServer.Endpoint
@Path("/orders")
class OrdersEndpoint {

    @Http.POST
    @AuthorizationPermission("ORDER_CREATE")
    String create(@Http.Entity CreateOrder request,
                  Principal principal) {
        return principal.getSubjectId();
    }
}
```

Kiev

- Provides Helidon configuration and service registry bindings for Kiev
- Supports in-memory, direct database, and Kiev-as-a-service backends
- Exposes `DataStore`, `MappedDataStore`, stream clients, and transaction support
- Adds `@KievTransaction` for declarative transaction handling

Kiev Configuration

- Config root is `oci.kiev`
- Each entry defines one named data store
- Store names qualify injected Kiev services and transactions
- `SERVICE` stores can expose stream records and include deleted column values

```
oci:
  kiev:
    data-stores:
      - backend: "IN_MEMORY"
        store-name: orders-store
        app-name: MyService
        transaction-max-reads: 100
        transaction-max-writes: 100
```

Kiev Example

- Inject `MappedDataStore` by configured store name
- Open mapped buckets from service code

```
@Service.Singleton
class OrderStore {

    @Service.Inject
    OrderStore(@Service.Named("orders-store")
               MappedDataStore dataStore) {
        var bucket = dataStore.getOrCreateBucket("orders",
                                                  String.class,
                                                  Order.class);
    }
}
```

Kiev Transactions

- `@KievTransaction` opens the transaction boundary around a service method
- The generated interceptor supplies a `Transaction` parameter when one is declared
- Successful write transactions commit on method exit
- Failures abort in-flight writes, then close the transaction

```
@KievTransaction(value = "orders-store", name = "order-put")
Order put(Transaction tx, Order order) {
    return bucket.put(tx, order);
}
```

Metrics

- Publishes Helidon meters to OCI Monitoring through `metrics-lib`
- Supports metrics recorded with OCI `Metrics`, Helidon APIs, or Helidon annotations
- Supports counters, timers, distribution summaries, gauges, and functional counters
- Adds automatic HTTP request metrics and built-in JVM gauges

Metrics Configuration

- Configured as a Helidon metrics publisher of type `oci`
- Project and fleet identify the emitted metric stream
- Region can be explicit or supplied through `oci-env`
- Endpoint can be derived from OCI Monitoring or explicitly overridden
- Include/exclude filters support exact, regex, or substring matching

```
metrics:  
  publishers:  
    - type: oci  
      project: my-service  
      fleet: my-fleet  
      region: us-ashburn-1  
      sample-interval: PT1M  
      metrics-scope-name: my-service  
      excludes: ["my-service\\.jvm\\\.\\.*."]  
      filter-matching-mode: regex
```


Metrics Example

- Service code uses Helidon metrics annotations
- Helidon Talon publishes metric updates to OCI metrics

```
@Http.GET
@Path("/hello/{name}")
@Metrics.Timed(value = "personalized-greeting",
               absoluteName = true)
String personalizedGreeting(@Http.PathParam("name") String name) {
    return "Hello, " + name + "!";
}
```

Object Storage

- Registers the synchronous OCI SDK `ObjectStorage` client for injection
- Builds it with shared OCI authentication and standard SDK client settings
- Routes to an explicit endpoint or a configured or injected OCI region
- Application code uses the native client and standard OCI requests

Object Storage Configuration

- Settings live under `oci.object-storage-client`
- `endpoint` overrides region-based routing
- Otherwise use `region`, `region-id`, or the injected OCI region; configure only one region key
- `client` controls timeouts, retry, circuit breaker, and upload behavior

```
oci:  
  object-storage-client:  
    region-id: us-ashburn-1  
    client:  
      connection-timeout: PT10S  
      read-timeout: PT1M
```

Object Storage Example

- Inject the OCI SDK `ObjectStorage` interface
- Build normal OCI SDK requests in service code

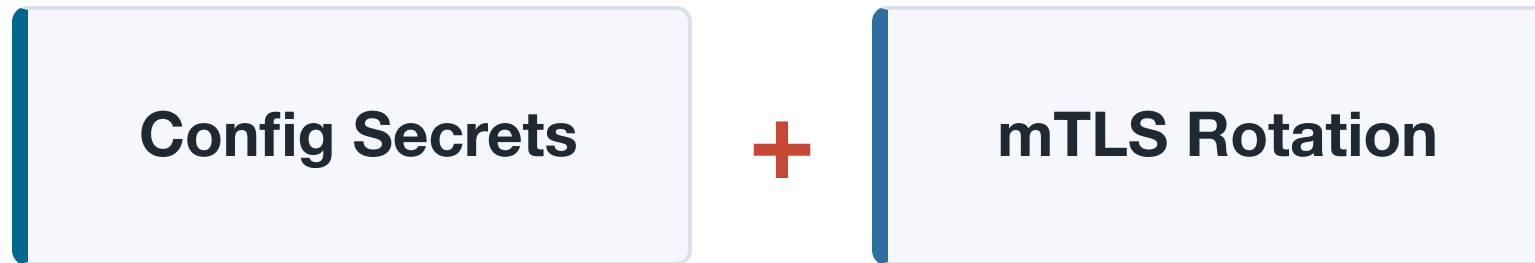
```
@Service.Singleton
class Buckets {
    private final ObjectStorage objectStorage;

    @Service.Inject
    Buckets(ObjectStorage objectStorage) {
        this.objectStorage = objectStorage;
    }

    void listBuckets(String namespace, String compartmentId) {
        objectStorage.listBuckets(ListBucketsRequest.builder()
            .namespaceName(namespace)
            .compartmentId(compartmentId)
            .build());
    }
}
```

Secret Service V2

- Exposes Secret Service V2 values through Helidon Config
- Provides server and client mTLS material



Secret Service V2 Secrets

- Exposes Secret Service V2 values through Helidon Config
- Maps a configured key prefix to SSv2 secret paths
- Resolves secrets lazily when application code reads the key
- Caches direct reads and polls only previously requested keys
- Can derive endpoints from environment configuration

Secret Service V2 Secrets Configuration

- Configured under `helidon.oci-secret-service`
- Prefix maps config lookups to Secret Service paths
- Endpoint can use `${oci.env.iaas-domain-name}`
- `poll-interval` defaults to `PT30M`; polling starts only with change listeners

```
helidon:
  oci-secret-service:
    prefix: "oci.ssv2"
    cache-ttl: "PT5M"
    poll-interval: "PT30M"
    client:
      endpoint: "https://secret-service-ce.${oci.env.iaas-domain-name}/v1"
```

Secret Service V2 Secrets Example

- Inject Helidon `Config`
- Read SSv2-backed values using the configured prefix

```
@Service.Singleton
final class SecretAwareService {
    private final String password;

    @Service.Inject
    SecretAwareService(Config config) {
        this.password = config.get("oci.ssv2/secret/orders/db/latest")
            .asString()
            .orElseThrow();
    }
}
```


Secret Service V2 mTLS

- Provides the `oci-ssv2` TLS manager
- Loads PKI JSON material from SSv2 or a Helidon resource
- Rotates server and client mTLS material on a reload schedule
- Supports Helidon clients and raw `SSLContext` use

Secret Service V2 mTLS Rotation

- Select `oci-ssv2` under Helidon TLS manager config
- `pki.secret-path` points to PKI JSON material in SSv2
- `reload.expression` controls the refresh cadence
- The same manager shape works for server and client TLS

```
server:
  tls:
    client-auth: "REQUIRED"
    manager:
      oci-ssv2:
        reload.expression: "0 0/30 * * * ? *"
        pki.secret-path: "/secret/orders/mtls/server/latest"
        trust.path: "/etc/oci-pki/ca-bundle.pem"
```

Workflow

- Integrates OCI Workflow-as-a-Service with Helidon Talon services
- Inject the native WFaaS `WorkflowClient` directly
- Use worker and poller clients through Helidon service registry names
- Configure endpoint, domain, timeouts, retry policy, and worker identity

Workflow Configuration

- Config root is `oci.workflow`
- Endpoint settings configure WFaaS routing and timeouts
- Worker and poller clients are exposed through service registry names

```
oci:
  workflow:
    domain-id: compute-control-plane
    endpoint-details:
      server-endpoint: https://wfaas.example.internal
      connect-timeout: PT5S
    worker-identifier: my-service-worker
```

Workflow Example

```
@Service.Singleton
class ProvisioningWorkflowService {
    private static final JsonBinding JSON = JsonBinding.create();
    private final WorkflowClient workflowClient;

    @Service.Inject
    ProvisioningWorkflowService(WorkflowClient workflowClient) {
        this.workflowClient = workflowClient;
    }

    WorkflowInstance launch(LaunchWorkflowRequest json) {
        byte[] payload = JSON.serialize(json)
            .getBytes(StandardCharsets.UTF_8);
        LaunchWorkflowArguments request = buildRequest(payload);
        return workflowClient.launchWorkflow(request);
    }
}
```

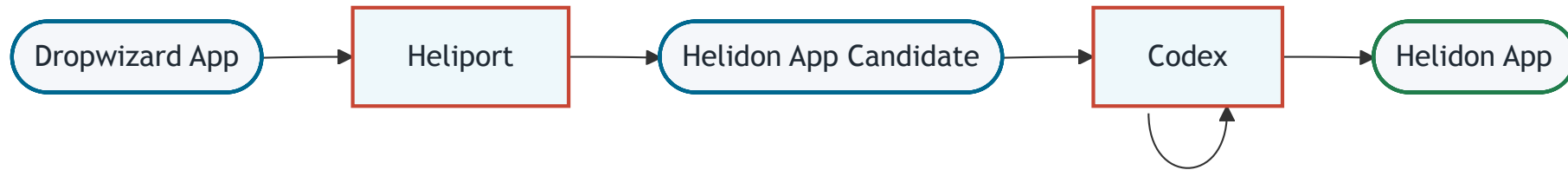
Integrated Client Versions

Client library	Managed version
OCI Java SDK	2.88.0
Helidon	4.5.1-M1
Identity Auth SDK	3.1.336
Kiev	2.0.35
Limits DP Java Client	1.9
Metrics module	2.0.12
Metrics library	1.1.30

Client library	Managed version
Workflow / WFaaS	14.0.0
Metering DP emitter	1.0.0.9
Metering CP agent	11.0.0.68
SPLAT SDK	0.1.39635
Tagging	4.3.5
Commons metrics	5.0.13
Vault user Java client	1.0.8

Heliport

- Migrates Dropwizard apps to Helidon Talon apps
- Combines OpenRewrite recipes with Codex-guided planning and validation
- Produces a migrated tree, `.heliport/` report, action plan, and proof evidence
- Still improving and can benefit from a Codex analysis after migration



Pilot Teams

16+

Pilot teams are looking to move existing services or build new services with Helidon Talon

**SPLAT
EDS
Cloud
Performance**

Active technical collaboration

**Secure Desktop
Clinical Config
Service**

Pilot services

More joining

The pilot pipeline continues to grow

Data Plane Reference App

- Runnable Helidon Talon reference service under `examples/data-plane`
- Combines hardened request ID handling, OCI error responses, Identity, Kiev, and Audit
- Demonstrates signed robot create/update/delete APIs with `@AuthorizationPermission`
- Uses in-memory Kiev by default so teams can run and test without external infrastructure

What's Next for Talon?

- Pegasus integration
- Enhanced logging support
- Control-plane reference app
- More integrations and features

What Teams Should Take Away

- Consider Heliport + Codex for Dropwizard-to-Helidon migration work
- Consult reference apps before wiring a new DP service
- Let `oci-env` resolve region, AD, FD, and domains
- Add only the Helidon Talon integrations your service needs
- Use the RC2 examples as executable configuration references

Resources

- User channel: `#helidon-users`
- Helidon: <https://helidon.io/>
- Documentation: <https://helidon.oraclecorp.com/docs/2.0.0-RC2/>
- Repository: <https://devops.oci.oraclecorp.com/devops-coderepository/namespaces/axuxirvibvvo/projects/HLDN/repositories/oci-helidon>
- Heliport: <https://devops.oci.oraclecorp.com/devops-coderepository/namespaces/axuxirvibvvo/projects/HLDN/repositories/heliport>